

Retriever Essentials Dashboard



CMSC 447

by Hader, Mosi, Raian, & Royce

The Goal of Our Project

Problem Statement:

UMBC's Retriever Essentials needs a centralized dashboard that aggregates multi-source data to replace manual tracking.

Purpose:

To help understand the impact and importance of their work.

Issue:

Existing software does not facilitate making data-driven insights on how basic needs support drives student retention and success.

SRS and Requirements Elicitation & Validation

Expectations: Develop a centralized dashboard for Retriever Essentials to combine data from multiple sources.

Our Interpretations: The stakeholders want metrics including percentages of involvement, which programs demonstrate the most popularity, and if involved students graduate at a higher rate.

Validation: We have validated our understanding of the stakeholders' expectations by attending stakeholder meetings and asking verifying questions.

Functional and Nonfunctional Requirements: Some functional requirements include a functional dashboard, pulling data into a single place, and The nonfunctional requirements include data upload rate and data querying speed.

Epics & User stories

Initial project backlog: Made Epics and User stories

Epics:

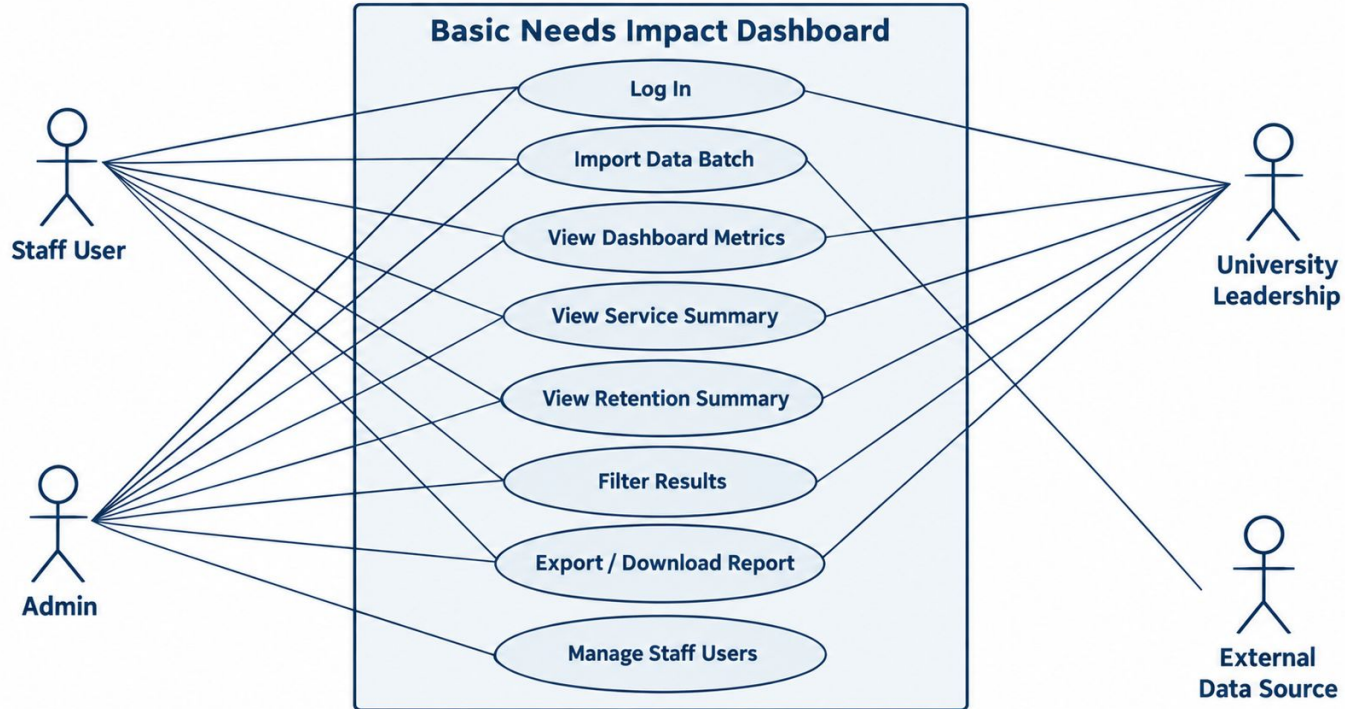
- Research
- Weekly meetings

User stories:

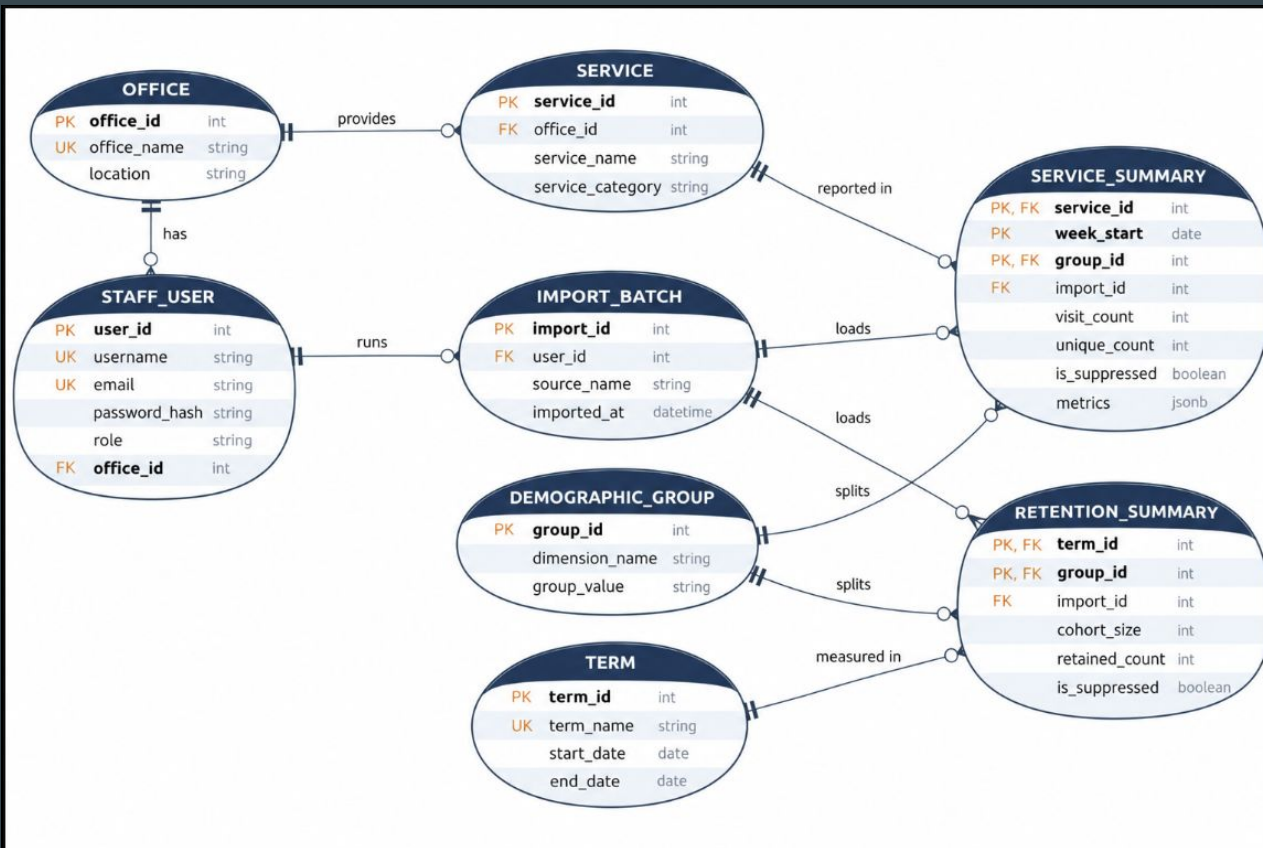
- Complete SRS
- Sprint 1 Presentation
- Setup team development environment

Use Case Diagram

Basic Needs Impact Dashboard - Use Case Diagram



ER Diagram



Sprint Planning

Sprint 1 - September 4-28th

Goal- Communicate and understand goal of stakeholders and how to start working on project

User Stories- Complete Proposal Presentation and SRS document, team development environment

Epics- Weekly meetings, research stakeholders needs

Stakeholders still deciding on what they want the final product to look like and what final ideas they have

Task Breakdown

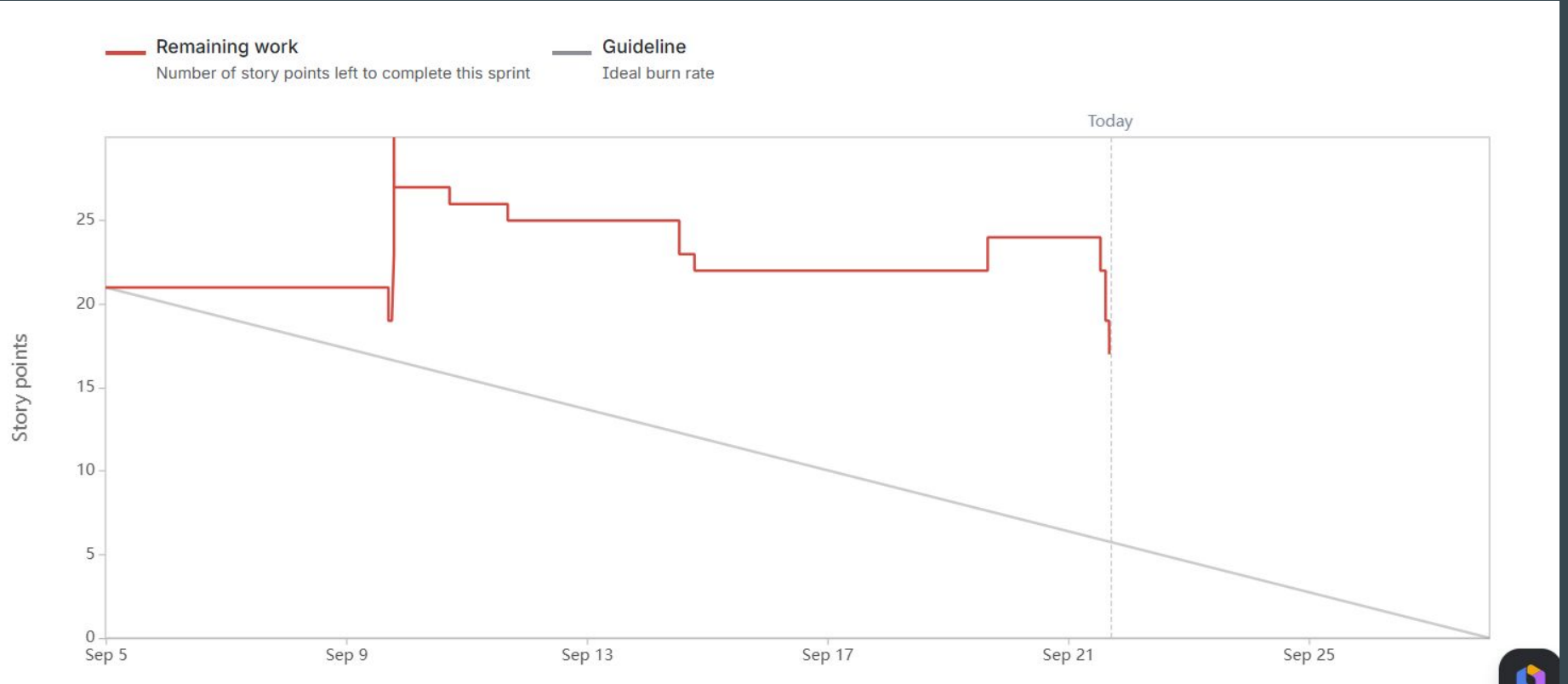
Task achievement- Tasks for sprint 1 are short and can be done anywhere from minutes to a day.

Organization- Most tasks can be unassigned since anyone can do them such as creating the powerpoint initially, or setting up github. Communicating works better than just assigning tasks.

Breakdown- Tasks are assigned to complete a bigger portion like a story. Ex- proposal presentation being split up and assigned.

Burn down chart- Skewed for first sprint since a lot of stuff can do at once since code and software has not been implemented. Mostly planning.

Burn Down Chart



Roles

Raian- Product Owner

Hader- Scrummaster

Royce- Developer

Mosi- Developer

Roles are lenient because tasks and creations are still decided through communicating

Ex- Mosi creation of google drives and documents

Hader- Creation of Jira board

Royce- Creation of GitHub

Raian- Splitting up parts and assigning tasks to team members

Collaboration Tools and Meetings

- Tools include Jira: Sprint tracking, GitHub: SRS and version control, Google Drive: notes and documents, Direct Messaging: Quick communication, and Discord: Online Meetings.
- Scheduling- Meetings on discord after Friday Stakeholder meetings.
- Stand ups- In person meetings often after class on Mondays and/or Wednesdays.
- Conflict resolution - Meeting together as a group to decide through asking questions to stakeholders or figuring out solutions between each other.

Technology stack

Frontend: We decided to use React for the front end for its flexibility in making the UI. This is also a very popular framework that our group personally wanted to work with.

Backend: We decided on using Python FastAPI as our backend. The Pydantic Library allows us to have validation before testing. We thought about using other APIs such as django for the self added security, but having the /docs endpoint built in for quick testing allows the people working on the front and back end to be able to understand the logic between their codes.

Database: We decided on using a relational database specifically Postgresql. This is because having a relational database with a fixed schema is critical for data integrity ensuring that invalid data cannot be added. And because the data will be aggregate before its received, we won't expect the data changing frequently.

Hosting & Deployment

- For Hosting, we will use docker compose to locally test and develop, so we all are on identical versions of everything when building
- For production, we will host it on AWS, using S3 to store the react files, our EC2 instance will run the backend, we will use RDS to host the database, and a secrets manager for tokens and internal secrets that we will need to setup manually, and we will use github actions to test pull requests.
- We will log health checks on the AWS instance as well as a billing alarm. Because some of the products we are using can become costly, we will integrate a cloudwatch alarm to monitor the costs of our project.

Stakeholder Interaction and Feedback Loop

- Retriever essentials will be the ones to provide the aggregated data, this will most likely be through an upload functionality we will implement
- University Leadership: They will be view only, to see overall trends in general
- Admin: Manage user access to the application
- First we will use the /docs for the backend demo, to ensure all logic is correct
- We have an integrated demo using the front and back end
- For usability testing we will have someone from retriever essentials admin and possibly a worker that has no ties to this project to test and do tasks on this application
- Compare data to ensure there are no mismatches
- Take feedback from the user including accuracy, ease of use, and other features wanted in the application
- In this feedback, we will add every form of bug, critique, etc to our jira board in order to properly track and execute our project

THANK YOU